Crash Recovery

Chapter 9

Failure Classification

Transaction failure :

- Logical errors: transaction cannot complete due to some internal error condition
- System errors: the database system must terminate an active transaction due to an error condition (e.g., deadlock)

System crash: a power failure or other hardware or software failure causes the system to crash.

- Fail-stop assumption: non-volatile storage contents are assumed to not be corrupted by system crash

Database systems have numerous integrity checks to prevent corruption of disk data

Failure Classification

Disk failure: a head crash or similar disk failure destroys all or part of disk storage

- Destruction is assumed to be detectable: disk drives use checksums to detect failures

Recovery Algorithms

Recovery algorithms are techniques to ensure database consistency and transaction atomicity and durability despite failures

- Recovery algorithms have two parts

1. Actions taken during normal transaction processing to ensure enough information exists to recover from failures

2. Actions taken after a failure to recover the database contents to a state that ensures atomicity, consistency and durability

Storage Structure

- Volatile storage:
  - does not survive system crashes
  - examples: main memory, cache memory
- Nonvolatile storage:
  - survives system crashes
  - examples: disk, tape, flash memory, non-volatile (battery backed up) RAM
- Stable storage:
  - a mythical form of storage that survives all failures
  - approximated by maintaining multiple copies on distinct nonvolatile media

Data Access

- Physical blocks are those blocks residing on the disk.
- Buffer blocks are the blocks residing temporarily in main memory.
- Block movements between disk and main memory are initiated through the following two operations:
  - input(B) transfers the physical block B to main memory.
  - output(B) transfers the buffer block B to the disk, and replaces the appropriate physical block there.
- Each transaction Ti has its private work-area in which local copies of all data items accessed and updated by it are kept.
  - Ti 's local copy of a data item X is called xi .
- We assume, for simplicity, that each data item fits in, and is stored inside, a single block.

Data Access

Transaction transfers data items between system buffer blocks and its private work-area using the following operations :

- read(X) assigns the value of data item X to the local variable xi .
- write(X) assigns the value of local variable xi to data item {X} in the buffer block.
- both these commands may necessitate the issue of an input(BX) instruction before the assignment, if the block BX in which X resides is not already in memory.

Data Access

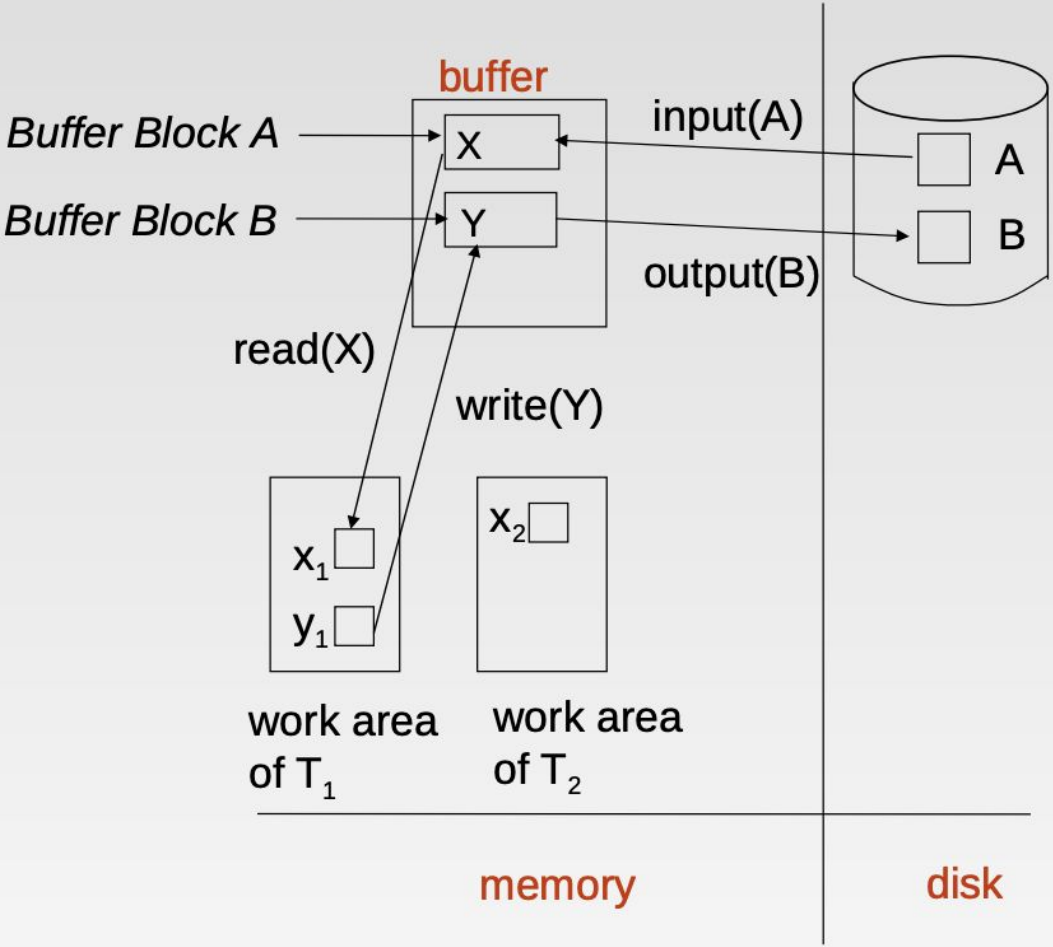
Transactions

- Perform read(X) while accessing X for the first time;
- All subsequent accesses are to the local copy.
- After last access, transaction executes write(X).

output(BX) need not immediately follow write(X).

System can perform the output operation when it deems fit.

Example of Data Access



Recovery and atomicity

Modifying the database without ensuring that the transaction will commit may leave the database in an inconsistent state.

- Consider transaction Ti that transfers $50 from account A to account B;

goal is either to perform all database modifications made by Ti or none at all.

- Several output operations may be required for Ti (to output A and B).

A failure may occur after one of these modifications have been made but before all of them are made.

Recovery and atomicity

■ To ensure atomicity despite failures, we first output information describing the modifications to stable storage without modifying the database itself.

■ We study two approaches:

- log-based recovery, and
- shadow-paging

■ We assume (initially) that transactions run serially, that is, one after the other.

Log Based Recovery

■ A log is kept on stable storage.

● The log is a sequence of log records, and maintains a record of update activities on the database.

■ When transaction Ti starts, it registers itself by writing a <Ti start>log record

■ Before Ti executes write(X), a log record <Ti, X, V1, V2> is written,
where V1is the value of X before the write,
and V2 is the value to be written to X.

● Log record notes that Ti has performed a write on data item Xj Xj
had value V1 before the write, and will have value V2 after the write.

Log Based Recovery

- When Ti finishes it last statement, the log record <Ti commit> is written.
- We assume for now that log records are written directly to stable storage (that is, they are not buffered)
- Two approaches using logs
    - Deferred database modification
    - Immediate database modification

Deferred Database Modification

■ The deferred database modification scheme records all modifications to the log, but defers all the writes to after partial commit.

■ Assume that transactions execute serially

■ Transaction starts by writing <Ti start> record to log.

■ A write(X) operation results in a log record <Ti, X, V> being written, where V is the new value for X

● Note: old value is not needed for this scheme

■ The write is not performed on X at this time, but is deferred.

■ When Ti partially commits, <Ti commit> is written to the log

■ Finally, the log records are read and used to actually execute the previously deferred writes.

Deferred Database Modification

■ During recovery after a crash, a transaction needs to be redone if and

only if both <Ti start> and<Ti commit> are there in the log.

■ Redoing a transaction Ti (redo Ti) sets the value of all data items update by the transaction to the new values.

■ Crashes can occur while

- the transaction is executing the original updates, or
- while recovery action is being taken

- example transactions T_0 and T_1 (T_0 executes before T_1):

T_0: **read** (A)	T_1 : **read** (C)
A: - A - 50	*C:- C- 100*
Write (A)	**write** (C)
read (B)	
B:- B + 50	
write (B)	

- Below we show the log as it appears at three instances of time.

(a)	(b)	(c)
<T_0 start>	<T_0 start>	<T_0 start>
<T_0, A, 950>	<T_0, A, 950>	<T_0, A, 950>
<T_0, B, 2050>	<T_0, B, 2050>	<T_0, B, 2050>
	<T_0 commit>	<T_0 commit>
	<T_1 start>	<T_1 start>
	<T_1, C, 600>	<T_1, C, 600>
		<T_1 commit>

- If log on stable storage at time of crash is as in case:
  - (a) No redo actions need to be taken
  - (b) redo(T_0) must be performed since <T_0 **commit**> is present
  - (c) **redo**(T_0) must be performed followed by redo(T_1) since <T_0 **commit**> and <T_i commit> are present

Immediate Database Modification

- The immediate database modification scheme allows database updates of an uncommitted transaction to be made as the writes are issued
  - since undoing may be needed, update logs must have both old value and new value
- Update log record must be written before database item is written
  - We assume that the log record is output directly to stable storage
  - Can be extended to postpone log record output, so long as prior to execution of an output(B) operation for a data block B, all log records corresponding to items B must be flushed to stable storage
- Output of updated blocks can take place at any time before or after transaction commit
- Order in which blocks are output can be different from the order in which they are written.

Immediate Database Modification Example

Log	Database
<T_0 **start**>	
<T_0, A, 1000, 950>	
T_0, B, 2000, 2050	
	A = 950 B = 2050
<T_0 **commit**>	
<T_1 **start**>	
<T_1, C, 700, 600>	
	C = 600
<T_1 **commit**>	

- Recovery procedure has two operations instead of one:
  - undo(Ti) restores the value of all data items updated by Ti to their old values, going backwards from the last log record for Ti
  - redo(Ti) sets the value of all data items updated by Ti to the new values, going forward from the first log record for Ti
- Both operations must be idempotent
  - That is, even if the operation is executed multiple times the effect is the same as if it is executed once Needed since operations may get re-executed during recovery
- When recovering after failure:
  - Transaction Ti needs to be undone if the log contains the record , but does not contain the record .
  - Transaction Ti needs to be redone if the log contains both the record and the record .
- Undo operations are performed first, then redo operations.

Below we show the log as it appears at three instances of time.

(a)	(b)	(c)
<T_0 start>	<T_0 start>	<T_0 start>
<T_0, A, 1000, 950>	<T_0, A, 1000, 950>	<T_0, A, 1000, 950>
<T_0, B, 2000, 2050>	<T_0, B, 2000, 2050>	<T_0, B, 2000, 2050>
	<T_0 commit>	<T_0 commit>
	<T_1 start>	<T_1 start>
	<T_1, C, 700, 600>	<T_1, C, 700, 600>
		<T_1 commit>

Recovery actions in each case above are:

(a) undo (T_0): B is restored to 2000 and A to 1000.

(b) undo (T_1) and redo (T_0): C is restored to 700, and then *A* and *B* are set to 950 and 2050 respectively.

(c) redo (T_0) and redo (T_1): A and B are set to 950 and 2050 respectively. Then *C* is set to 600

Shadow Paging

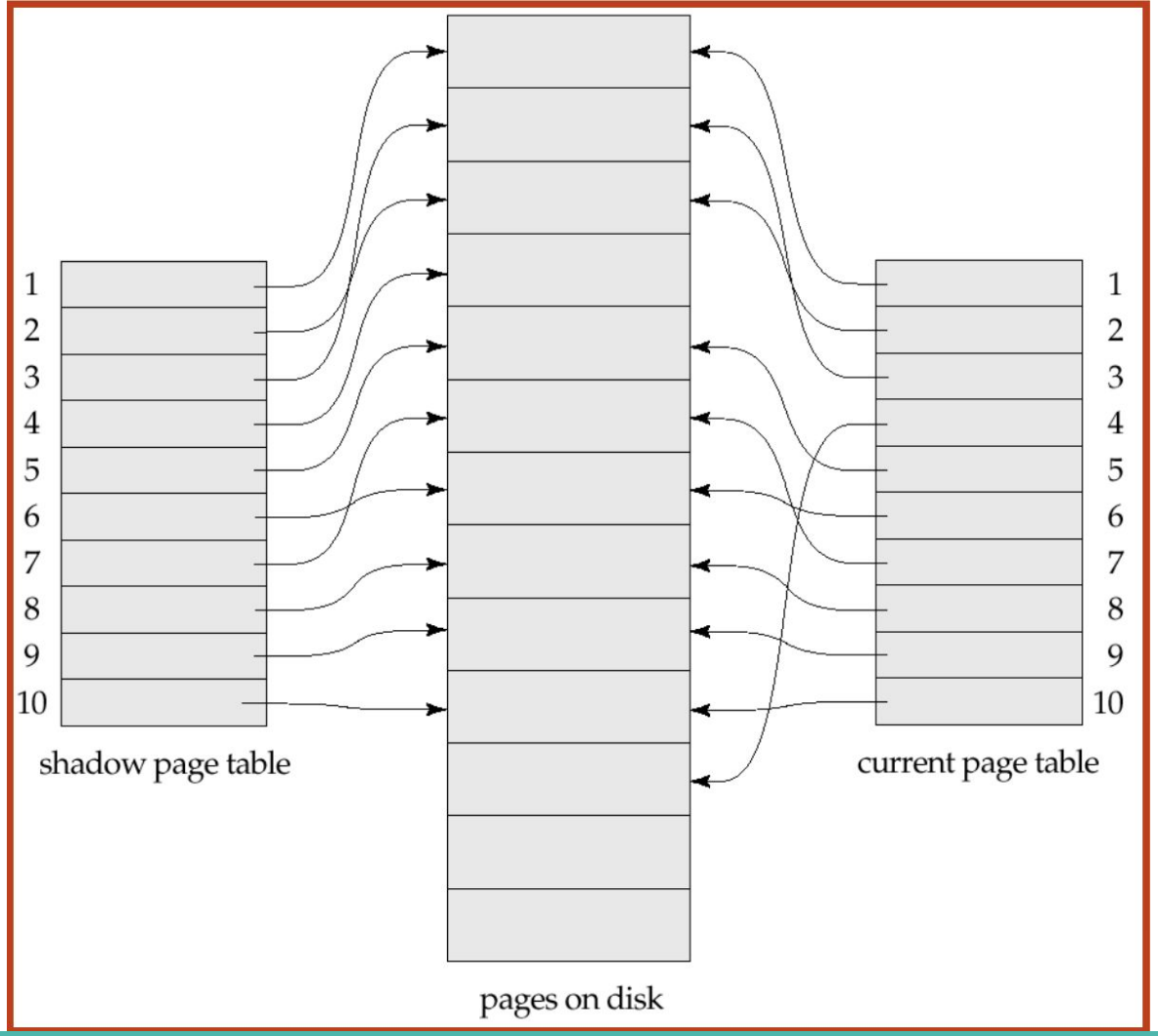
Shadow paging is a technique used in database management systems to provide atomicity and durability guarantees for transactions.

Here are some key points about it:

- Shadow paging maintains two copies of a database page - the current page and a shadow page.
- Modifications by transactions are first applied to the shadow page. The current page is marked "old" but remains unchanged.
- Once the transaction reaches its commit point, the changes are atomically applied to the database by making the shadow page the new current page.

Shadow Paging

- If the system crashes before commit, the original current page remains unchanged reflecting the pre-crash state. Changes to shadow pages are simply discarded.
- This ensures atomicity - either the entire transaction is reflected in the database or none of it is. There is no partial commit of a transaction.
- It also provides durability by preserving the pre-crash state in case of failure during a transaction.
- The "shadow" pages act as a temporary workspace isolated from other transactions to apply changes.


1
2
3
4
5
6
7
8
9
10
shadow page table
pages on disk
current page table
1
2
3
4
5
6
7
8
9
10

Upon successful transaction commit, the shadow pages containing the changes are atomically swapped and become the new current pages. The commit log is updated. The updated pages are now durable changes that reflect the committed transaction.

If the system crashes before transaction commit, upon restart, the system examines the commit log. Uncommitted transactions are identified and their corresponding shadow pages are discarded. This restores the original pre-crash database state by discarding uncommitted changes - achieving atomicity and durability.

The pages swapped by the last committed transaction just before the crash reflect durable persistent changes that have to be preserved upon recovery. Their changes are already on the current pages, not just shadow ones. These changes survive the crash and don't need special handling.

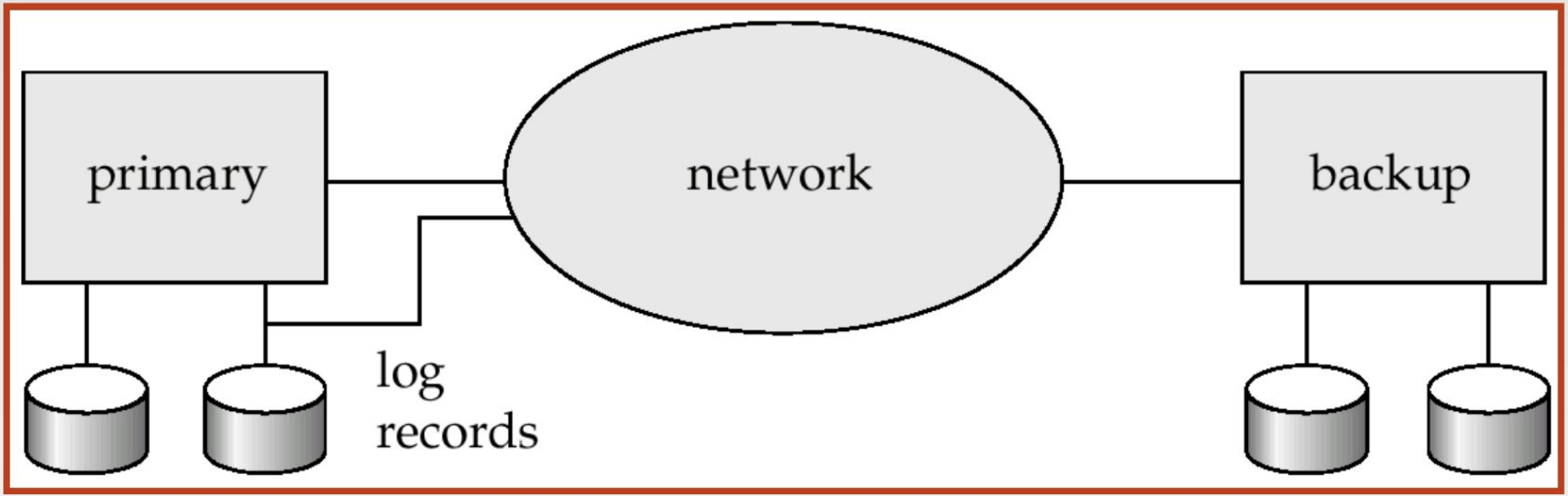
Advantages of Shadow paging

Advantages of shadow-paging over log-based schemes

- no overhead of writing log records
- recovery is trivial

Disadvantages of Shadow paging

- Expensive
- Commit Overhead
- Fragmentation
- Garbage Collection
- Algorithm extension

Remote Backup Systems

- Remote backup systems provide high availability by allowing transaction processing to continue even if the primary site is destroyed.